

Algorytmy optymalizacji dyskretnej, Lista 4

Błażej Pawluk

20 stycznia 2026

1 Zadanie 1

1.1 Opis problemu

Program ma generować k -wymiarową hiperkostkę z losową pojemnością krawędzi i znajdować w niej maksymalny przepływ między wierzchołkami.

1.1.1 Hiperkostka

k -wymiarową hiperkostką nazywamy graf skierowany $H_k = (N_k, A_k)$. Wierzchołków w grafie jest $|N_k| = 2^k$ i każdy wierzchołek ma jako etykietę i ciąg binarny o długości k , które interpretowane są jako liczby $e(i)$ od 0 do $2^k - 1$. Krawędzie tego grafu łączą wierzchołki różniące się liczbą jedynek w etykiecie o dokładnie 1, od mniejszej do większej, formalnie: $A_k = \{(i, j) \in N_k \times N_k : e(i) < e(j) \wedge H(i) - H(j) = 1\}$, gdzie $H(x)$ - waga Hamminga, czyli liczba jedynek w etykiecie. Pojemność każdej krawędzi u_{ij} losujemy jednostajnie ze zbioru $\{1, 2, \dots, 2^l\}$, gdzie $l = \max\{H(i), H(j), k - H(i), k - H(j)\}$.

1.1.2 Znajdowanie maksymalnego przepływu

Celem jest znalezienie maksymalnego przepływu między źródłem $s = 0$, a ujściem $t = 2^k - 1$. W tym celu skorzystamy z algorytmu Edmondsa-Karpa, czyli implementacji metody Forda-Fulkersona, polegającej na zwiększaniu przepływu po najkrótszych, co do liczby krawędzi ścieżkach powiększających.

1.2 Rozwiązanie

1.2.1 Generowanie hiperkostki

Uwagi:

Implementacja grafu przechowuje liczbę krawędzi n oraz listę sąsiedztwa, gdzie krawędź przechowywana jest w typie *Edge* z polami *to*, *capacity*, *flow*. Ponadto dodając krawędź przy pomocy funkcji *addEdge*(from, to, capacity) dodawane są dwie krawędzie jedna z *from* do *to* o pojemności *capacity* oraz przepływie 0 oraz odwrotną o pojemności 0 i przepływie 0.

Funkcje *ones* i *zeros* zwracają kolejno liczbę zer i jedynek w binarnej, k -znakowej reprezentacji liczby x .

Algorithm 1: Generowanie hiperkostki dla danego wymiaru k

```
1  $n \leftarrow 2^k$ 
2 for each vertex  $x$  from 0 to  $n$  do
3   for  $i = 0, 1, 2, \dots, k - 1$  do
4      $bit \leftarrow 2^i$ 
5     if  $x \& bit = 0$  then
6        $l \leftarrow \max\{ones(x), zeros(x), ones(x + bit), zeros(x + bit)\}$ 
7        $addEdge(\text{from: } x, \text{to: } x + bit, \text{capacity: } rand(1, 2^l))$ 
8     end
9   end
10 end
```

1.2.2 Algorytm Edmondsa-Karpa

Uwagi:

W zmiennej ap zliczamy liczbę ścieżek powiększających. Funkcja $reverse(e)$ zwraca krawędź w przeciwnym kierunku do krawędzi e (dodawanie takich krawędzi omówione w punkcie 1.2.1). Funkcja $BFS(s, t)$ szuka najkrótszej co do ilości krawędzi ścieżki z s do t w sieci rezydualnej, czyli grafie o tych samych wierzchołkach i krawędziach tam, gdzie $e.capacity - e.flow > 0$.

Algorithm 2: Implementacja algorytmu Edmondsa-Karpa

```
1  $ap \leftarrow 0$ 
2  $hasPath \leftarrow true$ 
3 while  $hasPath$  do
4    $p \leftarrow BFS(\text{from: } s, \text{to: } t)$ 
5    $cf \leftarrow \min\{e.capacity - e.flow \text{ for each edge } e \text{ in path } p\}$ 
6   for each edge  $e$  in path  $p$  do
7      $e.flow \leftarrow e.flow + cf$ 
8      $reverse(e) \leftarrow reverse(e).flow - cf$ 
9   end
10  $ap \leftarrow ap + 1$ 
11 end
```

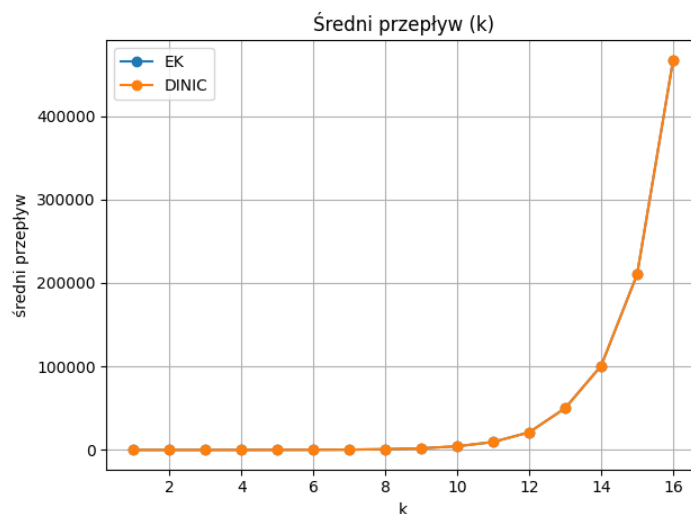
1.3 Złożoności

Tworzenie hiperkostki wykonywane jest w złożoności $O(n \times k) = O(k2^k)$. $BFS(s, t)$ przechodzi po wszystkich osiągalnych wierzchołkach i krawędziach, co daje: $O(V + E) = O(2^k + k \times 2^k) = O(k2^k)$. Znajdowanie kosztu wykonywane jest w złożoności $O(k)$ - porównujemy k elementów. Aktualizacja kosztów też jest wykonywana w $O(k)$, ponieważ długość ścieżki jest $O(k)$. Zgodnie z własnością algorytmu Edmondsa-Karpa pętla wykonywana jest $O(V \times E)$ razy (liczba ścieżek powiększających), co daje $O(2^k \times k2^k) = O(k2^{2k})$. Ostatecznie złożoność tego algorytmu to $O((V + E) \times V \times E) = O(V^2E + VE^2) = O(VE^2) = O(2^k \times (k2^k)^2) = O(k^22^{3k})$.

1.4 Eksperymenty

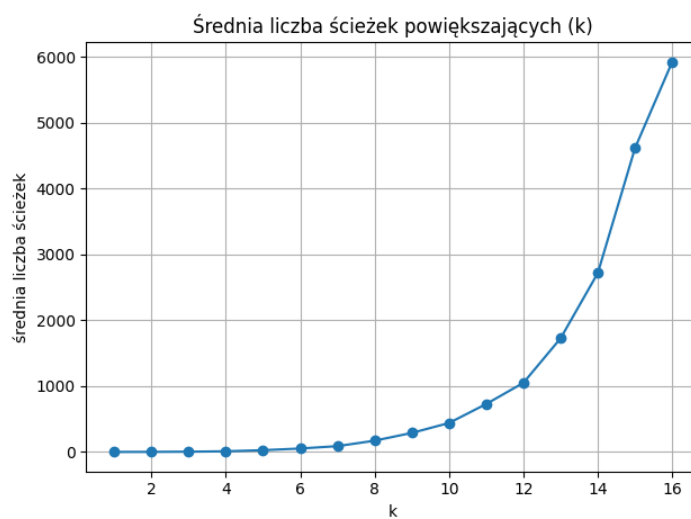
Eksperymenty wykonywane są dla $k = 1, 2, \dots, 16$ po 10 powtórzeń dla każdego k .

1.4.1 Średnia wielkość przepływu



Rysunek 1: Wyniki eksperymentów - średni przepływ dla różnych wartości wymiaru hiperkostki

1.4.2 Średnia liczba ścieżek powiększających



Rysunek 2: Wyniki eksperymentów - średnia liczba ścieżek powiększających dla różnych wartości wymiaru hiperkostki

1.4.3 Średni czas działania programu



Rysunek 3: Wyniki eksperymentów - średni czas wykonywania algorytmu dla różnych wartości wymiaru hiperkostki

1.5 Interpretacja i wnioski

Średni przepływ rośnie wykładniczo wraz z wymiarem kostki, co jest spodziewane - liczba wierzchołków, krawędzi i ich pojemności rośnie wykładniczo. Podobne wnioski można wyciągnąć z analizy wyników średniej liczby ścieżek powiększających. Średni czas działania programu potwierdza wyliczoną złożoność algorytmu - rośnie wykładniczo wraz ze wzrostem k .

2 Zadanie 2

2.1 Opis problemu

Program ma generować dwudzielny graf losowy, a następnie znajdować w nim największe skojarzenie.

2.1.1 Graf dwudzielny losowy

Generowany graf nieskierowany $G = (V_1 \cup V_2, E)$. Podzbiory V_1 i V_2 są rozłączne i mają moc 2^k . Z każdego wierzchołka wychodzi i krawędzi do niezależnie i jednostajnie losowo wybranych wierzchołków z V_2 .

2.1.2 Największe skojarzenie

Do wygenerowanego grafu dodajemy 2 wierzchołki: źródło S oraz ujście T . Graf zmieniamy na skierowany - każdą krawędź zamieniamy na krawędź z V_1 do V_2 o pojemności 1. Dodajemy również krawędzie z S do każdego wierzchołka z V_1 oraz z każdego wierzchołka z V_2 do T o pojemności 1. Następnie szukając największy przepływ w tak skonstruowanym

grafie otrzymujemy największe skojarzenie - przez pojemności równe 1, żadna z krawędzi nie zostanie użyta dwukrotnie - otrzymamy skojarzenie o największej ilości krawędzi.

2.2 Rozwiązanie

2.2.1 Generowanie grafu dwudzielnego losowego

Uwagi:

Implementacja grafu zgodna z poprzednim zadaniem.

Algorithm 3: Generowanie grafu dwudzielnego losowego dla danego rozmiaru k oraz stopnia i

```

1  $n \leftarrow 2^{k+1} + 2$  // 2 zbiory o rozmiarze  $2^k$  plus źródło i ujście
2 for each vertex  $v$  in  $V_1$  do
3    $\text{addEdge}(\text{from: } T, \text{ to } v, \text{ capacity: } 1)$ 
4    $u \leftarrow \text{random vertex in } V_2$ 
5    $\text{addEdge}(\text{from: } v, \text{ to } u, \text{ capacity: } 1)$ 
6 end
7 for each vertex  $v$  in  $V_2$  do
8    $\text{addEdge}(\text{from: } v, \text{ to } S, \text{ capacity: } 1)$ 
9 end

```

2.2.2 Szukanie skojarzenia

Znajdowanie skojarzenia sprowadza się do znalezienia największego przepływu w grafie skonstruowanym wcześniej. Do wypisania tego skojarzenia wystarczy wypisać krawędzie, dla których pole *flow* jest równe 1.

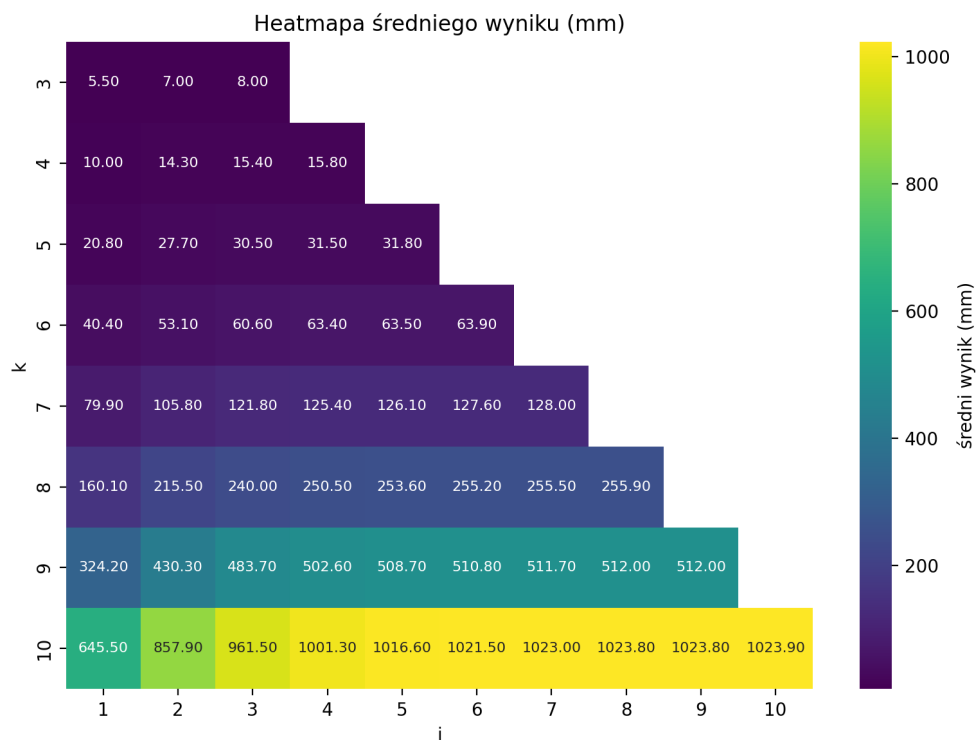
2.3 Złożoności

Tworzenie grafu wykonywane jest w $O(V) = O(2^{k+1} + 2) = O(2^k)$. Znajdowanie skojarzenia wykonuje algorytm Edmondsa-Karpa wykonywany jest w złożoności $O(VE^2) = O((2^{k+1} + 2) \times ((i + 2)2^k)^2) = O(i^2 2^{3k})$.

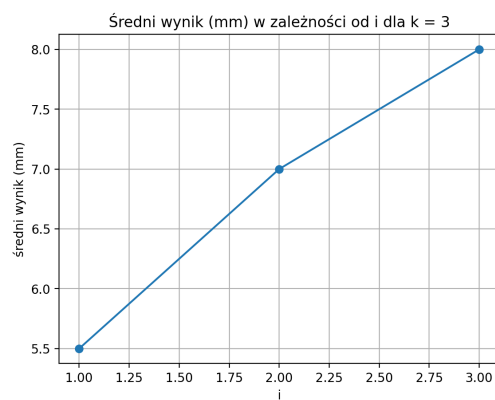
2.4 Eksperymenty

Eksperymenty były wykonywane dla $k = 3, 4, \dots, 10$ oraz $i = 1, 2, \dots, k$ po 10 powtórzeń dla każdej kombinacji parametrów.

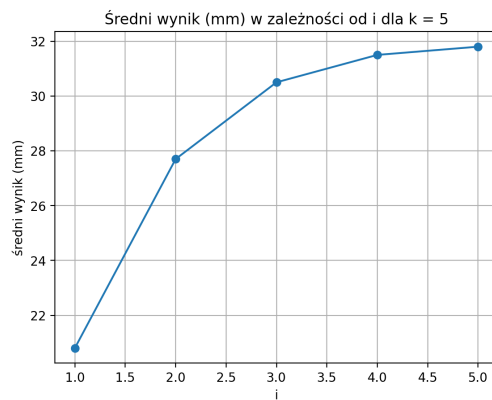
2.4.1 Średni rozmiar maksymalnego skojarzenia



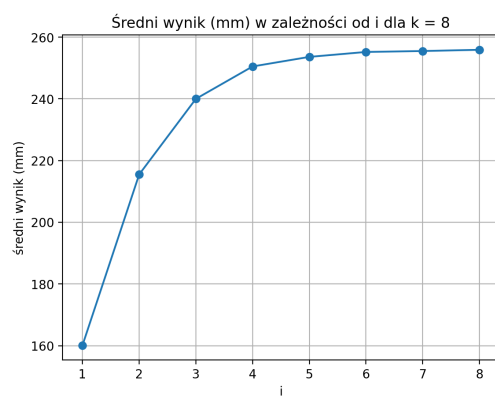
Rysunek 4: Heatmapa przedstawiająca średni rozmiar w zależności od rozmiaru k oraz stopnia i



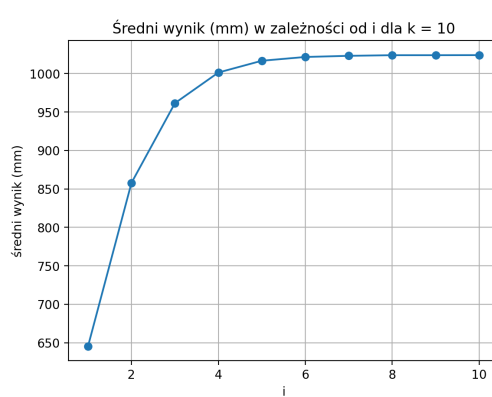
(a)



(b)



(c)



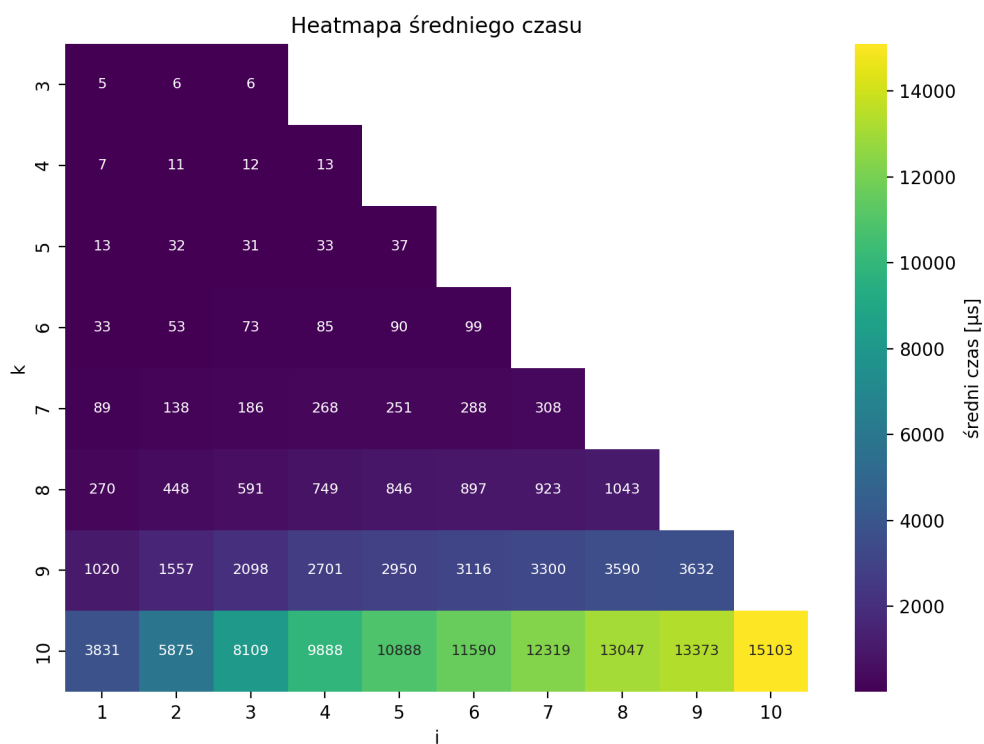
(d)

Rysunek 5: Wykresy średnich rozmiarów skojarzenia dla rozmiarów $k = 3, 5, 8, 10$ w zależności od stopnia i



Rysunek 6: Wykres średnich rozmiarów skojarzenia w zależności od rozmiaru k oraz stopnia $i = k$

2.4.2 Średni czas działania programu



Rysunek 7: Heatmapa przedstawiająca średni czas działania programu w zależności od rozmiaru k oraz stopnia i



Rysunek 8: Wykres średnich czasów działania programu w zależności od rozmiaru k oraz stopnia $i = k$

2.5 Interpretacja i wnioski

Średnie rozmiary skojarzenia zależą zarówno od rozmiaru k jak i stopnia i . Wykresy wyników rozmiaru w zależności od stopnia i pokazują, że rozmiar rośnie logarytmicznie wraz z i dla danego k . Natomiast gdy stopień jest stale ustawiony na $i = k$, to rozmiar rośnie wykładniczo. Wyniki nigdy jednak nie przekraczają wartości 2^k , ponieważ mając 2×2^k wierzchołków nie możemy znaleźć skojarzenia większego niż połowa liczby wierzchołków grafu. Czas działania programu rośnie wykładniczo, co potwierdza wyliczoną złożoność programu.

3 Zadanie 3

3.1 Opis zadania

Dodanie do poprzednich programów opcję wygenerowania kodu programu tworzącego model programowania liniowego rozwiązujący ten sam problem w modelu GLPK.

3.2 Rozwiązanie

Przy wywołaniu programów z odpowiednią flagą, program generuje plik z kodem w języku `julia` tworzący i rozwiązujący model programowania liniowego rozwiązujący ten sam problem dla tego samego grafu.

3.2.1 Maksymalny przepływ wyliczany algorytmem Edmondsa-Karpa

Parametry: x_{ij} - przepływ na krawędzi między wierzchołkami i oraz j , tylko tam, gdzie faktycznie jest krawędź w grafie oraz krawędzie odwrotne

Ograniczenia: dla każdego wierzchołka dodajemy ograniczenie górne na maksymalny przepływ na krawędzi. Dodajemy też n ograniczeń na bilans: dla wszystkich wierzchołków z grafu (poza źródłem i ujściem) suma przepływu przychodzącego i wychodzącego musi być równa.

Funkcja kosztu: maksymalizujemy sumę przepływów wychodzących ze źródła.

3.2.2 Maksymalne skojarzenie

Szukając skojarzenia korzystamy z tego samego modelu.

3.3 Eksperymenty

Wykonałem po kilka powtórzeń dla różnych wartości parametrów wejściowych w obu programach. Wartości maksymalnego przepływu oraz wielkości maksymalnego skojarzenia były takie same. Zdecydowanie szybciej rozwiązanie liczyły algorytmy pisane przeze mnie, niż modele programowania liniowego. Przepływy po wszystkich łukach i maksymalne skojarzenia nie zawsze są takie same. Przykłady wywołań poniżej:

```

$ ./EdmondsKarp --size 4 --printFlow --glpk ek.jl
===== INPUT =====
Performing algorithm for:
  k: 4
  printFlow: yes
  glpk: yes
  filename: ek.jl
=====

===== OUTPUT =====
      max flow: 27
-----
(0,1)=3/3
(0,2)=7/7
(0,4)=7/7
(0,8)=10/10
(1,3)=1/4
(1,5)=0/2
(1,9)=2/2
(2,3)=2/2
(2,6)=4/5
(2,10)=1/1
(3,7)=0/5
(3,11)=3/7
(4,5)=1/1
(4,6)=1/3
(4,12)=5/5
(5,7)=0/4
(5,13)=1/1
(6,7)=3/5
(6,14)=2/2
(7,15)=3/3
(8,9)=6/6
(8,10)=4/4
(8,12)=0/1
(9,11)=2/2
(9,13)=6/8
(10,11)=5/7
(10,14)=0/1
(11,15)=10/10
(12,13)=5/6
(12,14)=0/1
(13,15)=12/12
(14,15)=2/3
-----
Wygenerowano kod języka julia w pliku 'ek.jl'.
===== OUTPUT P2 =====
      execution time: 0
      augmentic paths: 12

```

(a)

```

$ julia ek.jl
max flow: 27
time: 1.590991
flow:
      (0,1)=3/3
      (0,2)=7/7
      (0,4)=7/7
      (0,8)=10/10
      (1,3)=3/4
      (1,5)=0/2
      (1,9)=0/2
      (2,3)=2/2
      (2,6)=4/5
      (2,10)=1/1
      (3,7)=0/5
      (3,11)=5/7
      (4,5)=1/1
      (4,6)=1/3
      (4,12)=5/5
      (5,7)=0/4
      (5,13)=1/1
      (6,7)=3/5
      (6,14)=2/2
      (7,15)=3/3
      (8,9)=5/6
      (8,10)=4/4
      (8,12)=1/1
      (9,11)=0/2
      (9,13)=5/8
      (10,11)=5/7
      (10,14)=0/1
      (11,15)=10/10
      (12,13)=5/6
      (12,14)=1/1
      (13,15)=11/12
      (14,15)=3/3

```

(b)

Rysunek 9: Wynik programu szukania maksymalnego przepływu algorytmem Edmondsa-Karpa (po lewej) oraz modelem programowania liniowego (po prawej). Maksymalny przepływ taki sam, natomiast dokładny przepływ po łukach się różni (np. krawędź (1,3)).

```

$ ./Matching --size 3 --degree 3 --printMatching --glpk m.jl
===== INPUT =====
Performing algorithm for:
  k: 3
  i: 3
  printMatching: yes
=====
===== OUTPUT =====
  max matching: 8
-----
(1,10)
(2,16)
(3,12)
(4,14)
(5,15)
(6,11)
(7,9)
(8,13)
-----
Wygenerowano kod języka julia w pliku 'm.jl'.
===== OUTPUT P2 =====
execution time: 0

```

(a)

```

$ julia m.jl
matching: 8
time: 1.6175853
matching:
  (1,14)
  (2,9)
  (3,10)
  (4,11)
  (5,15)
  (6,12)
  (7,16)
  (8,13)

```

(b)

Rysunek 10: Wynik programu szukania maksymalnego skojarzenia algorytmem Edmondsa-Karpa (po lewej) oraz modelem programowania liniowego (po prawej). Rozmiar maksymalnego skojarzenia taki sam, natomiast dokładne skojarzenie się różni (np. krawędź (1,10) obecna w rozwiązaniu algorytmem Edmondsa-Karpa, ale nieobecna w rozwiązaniu modelem programowania liniowego).

3.4 Interpretacja i wnioski

Modele programowania liniowego takie jak GLPK są przydatnym narzędziem umożliwiającym rozwiązywanie szerokiej gamy problemów. Nie zawsze są one jednak najlepszym rozwiązaniem. Na naszym przykładzie: algorytm Edmondsa-Karpa liczy rozwiązanie dla małych danych niemal natychmiast, natomiast model GLPK liczy je o wiele dłużej (*0ms* Edmondsem-Karpem vs. *1.62s* modelem przy liczeniu skojarzeń).

4 Zadanie 4 - dodatkowe

4.1 Opis problemu

Rozwiązujemy ten sam problem co w zadaniu 1: szukamy największego przepływu w losowo wygenerowanej hiperkostce. Tym razem jednak będziemy korzystali z algorytmu Dinica, który wykorzystuje struktury *level graph* oraz *blocking flow*, aby otrzymać lepszą złożoność, niż w algorytmie Edmondsa-Karpa.

4.2 Rozwiązanie

Uwagi:

Graf jest zaimplementowany w ten sam sposób, co w poprzednich zadaniach. W algorytmie korzystamy ze struktury *level*, która jest *level graphem*. Jest to graf $L = (V, E_L)$, gdzie $E_L = \{(u, v) \in E : (u, v).capacity - (u, v).flow > 0 \wedge dist(u) + 1 = dist(v)\}$, czyli zawiera tylko krawędzie z sieci rezydualnej, które przybliżają nas do ujścia t . Do jego wyznaczania używamy funkcji $BFS(s, t)$. Następnie szukamy przepływu blokującego w grafie *level*, czyli przepływu, w którym każda ścieżka z s do t zawiera w sobie conajmniej jedną krawędź e taką, że $e.capacity - e.flow = 0$. Dopóki takie przepływu nie ma - powiększamy przepływ o nowe ścieżki znalezione funkcją $DFS(s, t)$, która zwraca ścieżkę

i przepływ, o który ta ścieżka powiększa łączny przepływ. Po znalezieniu takiej ścieżki dopiero wykonujemy ponownie $BFS(s, t)$.

Algorithm 4: Implementacja algorytmu Dinica

```

1  $f \leftarrow 0$ 
2  $level = BFS(s, t)$ 
3 while  $level$  has path from  $s$  to  $t$  do
4   while  $path, p \leftarrow DFS(s, t)$  exists do
5     for each vertex  $v$  in  $path$  do
6        $v.flow \leftarrow p$ 
7        $reverse(e).flow \leftarrow reverse(e).flow - p$ 
8      $f \leftarrow f + p$ 
9   end
10 end
11  $level \leftarrow BFS(s, t)$ 
12 end
13 return  $f$ 

```

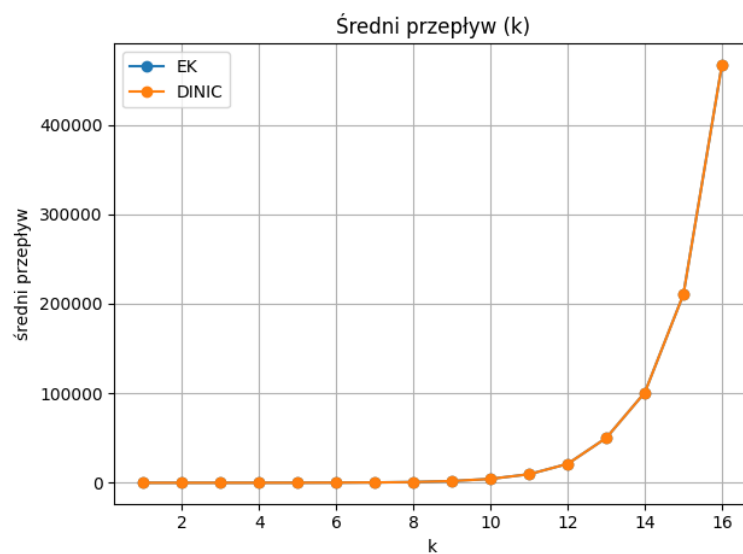
4.3 Złożoność

Wykonanie $BFS(s, t)$ do zbudowania $level$ wykonywane jest w $O(E) = O(k2^k)$. Taka sama złożoność jest dla funkcji $DFS(s, t)$ zoptymalizowanej pod ten przypadek. Główna pętla wykonywana $O(V)$ razy. Zatem łączna złożoność algorytmu wynosi $O(VE) = O(2^k \times k2^k) = O(k2^k)$.

4.4 Eksperymenty

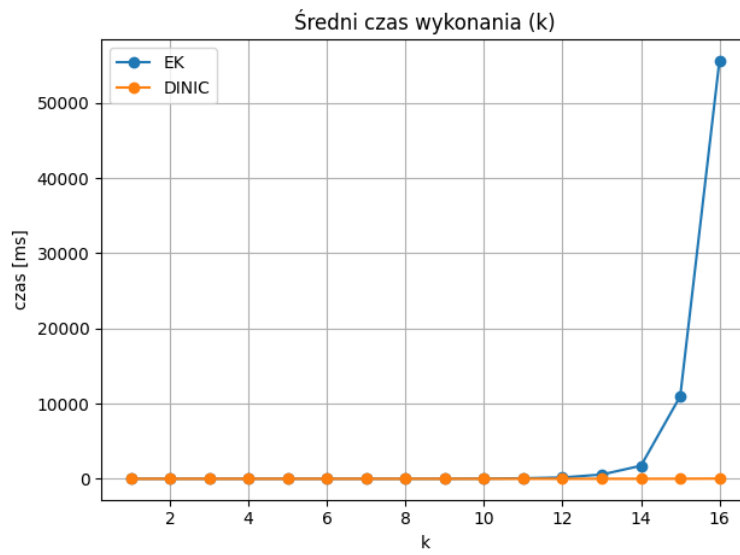
Eksperymenty wykonywane dla $k = 1, 2, \dots, 16$ po 10 powtórzeń dla obu algorytmów.

4.4.1 Porównanie wyników



Rysunek 11: Porównanie wyników średniego maksymalnego przepływu dla algorytmów Edmondsa-Karpa i Dinica (widoczna jedna seria danych - wyniki są takie same).

4.4.2 Porównanie średnich czasów



Rysunek 12: Porównanie średnich czasów wykonywania programów algorytmem Edmondsa-Karpa i Dinica (nie wliczany czas generowania wykresów).

4.4.3 Średnie czasy dla algorytmów Dinica



Rysunek 13: Średni czas wykonywania algorytmu Dinica w zależności od rozmiaru k .

4.5 Interpretacja i wyniki

Algorytmy Dinica i Edmondsa-Karpa zwracają dokładnie takie same wyniki maksymalnego przepływu. Algorytm Dinica ma jednak o wiele lepszą złożoność. Czas średniego czasu potwierdza wyliczoną złożoność.